CHAPTER 1: INTRODUCTION

1.1 Project Background & Industry Context

The global automotive mobility and car rental industry has witnessed exponential technological transformation over the past decade. Traditionally dominated by brick-and-mortar storefronts, physical paper rental agreements, and manual reservation logbooks, the market is rapidly pivoting toward on-demand digital fleet platforms. Consumers today expect instant online reservations, transparent real-time vehicle availability, multi-tier duration discounts, and seamless digital payment execution directly from their web and mobile browsers.

In urban centers and corporate executive sectors, there is a burgeoning demand for specialized luxury car rentals, zero-emission electric supercars (such as Porsche Taycan and Tesla Model S), and certified chauffeur-driven VIP services. However, independent fleet operators and boutique rental agencies frequently struggle to compete due to outdated software infrastructure. "AutoDrive" was conceived to bridge this technological divide by providing a robust, high-performance, and visually captivating Automotive Fleet SaaS Web Application that automates the complete rental lifecycle from vehicle discovery to verified return.

1.2 Problem Statement

A comprehensive analysis of prevailing automotive rental management systems reveals critical operational flaws that negatively impact business revenue and customer satisfaction:

- **Double-Booking Catastrophes:** Many smaller rental systems rely on loose availability flags or manual checks. If two customers attempt to rent the same vehicle for overlapping time slots, systems fail to detect concurrency collisions, leading to unfulfilled bookings, severe brand erosion, and customer disputes.
- **Chauffeur Allocation Conflicts:** Existing platforms decouple vehicle booking from driver dispatch. When a customer requests a chauffeur-driven rental, drivers are often assigned manually via phone calls, resulting in driver double-booking during peak holiday seasons.
- **Opaque & Inflexible Pricing:** Rental pricing is often static or calculated manually. Customers lack clear visibility into duration-based tiered savings (such as automated discounts for week-long rentals) and extra add-on charges (such as commercial chauffeur fees or comprehensive insurance).
- **Payment Gateway Simulation Bottlenecks:** Integrating production payment gateways (such as Razorpay, Stripe, or PayPal) requires strict business KYC verification, GST registration, and bank approvals. In testing and academic environments, public test keys frequently disable payment methods on live servers, displaying 'No appropriate payment method found' errors and crippling developer testability.
- **Dated User Interfaces:** Most enterprise car rental interfaces resemble cluttered database entry portals. They lack modern design aesthetics, mobile responsiveness, intuitive comparison matrices, and clear contrast.

1.3 Objectives of the Project

The primary objective of AutoDrive is to develop an enterprise-grade, conflict-free Car Rental System that solves these problems through modern web engineering. Specifically, the project achieves:

- **Mathematical Slot Collision Prevention:** Implement an automated Date Range Overlap Collision Detection Engine using Django ORM Q-queries (start_date <= end_date and end_date >= start_date) to guarantee that no vehicle or driver can be booked twice in the same time window.
- **Dynamic Multi-Day Tiered Pricing:** Provide real-time pricing calculations that automatically deduct 10% for rentals of 3+ days and 25% for rentals of 7+ days, with instantaneous client-side UI recalculation.
- **Integrated Chauffeur Scheduling:** Support both Self-Drive and Professional Chauffeur modes with automatic date-conflict filtering for assigned drivers.
- **Dual-Mode Razorpay Payment Gateway:** Provide seamless integration with the Razorpay Standard Checkout SDK alongside a built-in 3D Secure Bank Simulator with Success/Decline simulation for 100% reliable local testing.
- **Administrative Fleet Intelligence:** Equip fleet managers with real-time revenue analytics, active utilization metrics, chauffeur dispatch schedules, and booking lifecycle controls.

1.4 Scope of the Project

The functional scope of AutoDrive covers:

- **Customer Portal:** User registration, authentication, multi-facet fleet catalog search (by category, transmission, fuel type, horsepower), vehicle comparison tool, live date picker reservation, chauffeur selection, digital checkout, booking receipts, and verified customer reviews.
- **Chauffeur Subsystem:** Chauffeur profiles, commercial license management, experience badges, rating histories, and daily fee calculations.
- **Payment Subsystem:** Credit/Debit Cards, UPI QR codes, Netbanking, Concierge Wallet, and Razorpay Bank Simulator.
- **Administrative Subsystem:** Role-based administrative dashboard, vehicle fleet management (CRUD), reservation status lifecycle management (Pending -> Confirmed -> Completed -> Cancelled), and financial analytics.

1.5 Organization of the Report

This project report is structured into seven comprehensive chapters: Chapter 1 introduces the project background, problem statement, objectives, and scope. Chapter 2 reviews literature and comparative studies. Chapter 3 details functional, non-functional, hardware, and software requirements. Chapter 4 presents system architecture, DFDs, ERDs, database designs, and UML diagrams. Chapter 5 covers implementation modules and code snippets. Chapter 6 details the testing strategy, test cases, and results. Finally, Chapter 7 concludes the report with commercial roadmaps and future enhancements.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview of Modern Car Rental Platforms

The evolution of automotive rental software has progressed across three distinct paradigms:

- **Traditional Legacy Systems (Avis, Hertz, Enterprise):** Pioneered computerized reservation systems in the 1980s. While robust for massive airport operations, they rely on complex mainframe backends, exhibit slow UI responsiveness, and lack flexibility for boutique luxury rentals.
- **Peer-to-Peer (P2P) Marketplaces (Turo, Getaround):** Empower individual car owners to list private vehicles. While offering diverse vehicle models, they suffer from inconsistent vehicle maintenance, unpredictable host reliability, and complex insurance arbitration.
- **Modern On-Demand Fleets (Zoomcar, Zipcar):** Feature mobile-first self-drive bookings. However, they frequently experience slot allocation glitches during surge periods and lack integrated certified executive chauffeur options.

2.2 Comparative Analysis of Existing Systems

Table 2.1 presents a comparative feature matrix benchmarking AutoDrive against existing market solutions.

Feature / Capability	Legacy Counters	P2P (Turo)	On-Demand (Zoomcar)	AutoDrive CRS (Proposed)
Real-Time Slot Conflict Engine	Manual / Partial	Host Dependent	Algorithmic	Strict Mathematical ORM Guarantee
Certified Chauffeur Dispatch	External Add-on	Not Supported	Not Supported	Native Integrated Chauffeur Engine
Multi-Day Tiered Discounts	Coupon Codes Only	Host Discretion	Variable Surge	Automated Tiers (10% & 25%)
Payment Gateway Sandbox	N/A (Production Only)	N/A	N/A	Dual-Mode (SDK + Bank Simulator)
Vehicle Comparison Matrix	Limited	Basic Side-by-Side	None	Full Multi-Spec Matrix
Admin Intelligence Dashboard	Complex ERP	Basic Host App	Proprietary	Real-Time SaaS Visual Metrics

UI/UX Design Language	Legacy Enterprise	Consumer Web	Mobile Web	Cyberpunk-Luxury Dark Glassmorphism

2.3 Research Gaps & Proposed Engineering Solutions

The comparative study identifies three distinct engineering gaps that AutoDrive resolves:

- **Atomic Slot Booking Isolation:** Most lightweight platforms execute naive availability checks ('car.is_available == True'). If a car is booked next week, it is marked unavailable today, or conversely, overlapping bookings are permitted. AutoDrive implements mathematical interval overlap checking (StartA <= EndB and EndA >= StartB) within atomic database transactions.
- **Decoupled Payment Gateway Simulation:** Educational and staging environments often break when external payment APIs require live KYC. AutoDrive implements a decoupled Bank Simulator mimicking Razorpay's exact 3D Secure modal, enabling realistic development without merchant account dependencies.
- **Unified Fleet & Driver Scheduling:** AutoDrive synchronizes driver schedules alongside vehicle reservations, ensuring that certified chauffeurs are not double-booked across separate vehicles.

2.4 Summary

The literature review demonstrates that AutoDrive addresses the primary shortcomings of both legacy and modern rental platforms by combining strict concurrency validation, dynamic pricing, integrated chauffeur scheduling, and robust payment simulation into a streamlined, high-performance web architecture.

CHAPTER 3: SYSTEM REQUIREMENT ANALYSIS

3.1 Functional Requirements

Functional requirements define the core capabilities, operations, and services the system must deliver. Table 3.1 details the functional requirements matrix for AutoDrive.

Req. ID	Module	Requirement Description	Input / Trigger	Expected Output
FR-01	Auth	Role-Based Authentication	Username, Password	Session token, role redirection (Customer/Admin)
FR-02	Fleet	Multi-Facet Fleet Discovery	Category, Fuel, Gearbox, Search	Filtered car cards with live pricing
FR-03	Detail	Live Multi-Day Rate Calculator	Start Date, End Date	Dynamic subtotal with 10%/25% discounts
FR-04	Conflict	Vehicle Slot Collision Check	Start Date, End Date	Validation error if dates overlap existing booking
FR-05	Chauffeur	Certified Driver Scheduling	Chauffeur Dropdown Selection	Driver conflict check; daily chauffeur fee addition
FR-06	Gateway	Razorpay Standard Checkout	Click Proceed to Payment	Dual-mode popup: Live SDK or Bank Simulator
FR-07	Simulator	Bank 3D Secure OTP Simulation	Simulate Success / Failure	HTTP 200 approval or simulated decline
FR-08	Receipt	Digital Invoice Generation	Completed Booking Ref	Printable tax invoice with transaction ID
FR-09	Customer	My Bookings Hub	User Login Session	Chronological list of Active & Past Bookings
FR-10	Admin	Fleet Business Dashboard	Staff Authentication	Total revenue, fleet count, active trip controls

3.2 Non-Functional Requirements

- **Usability:** The user interface utilizes high-contrast glassmorphism, responsive CSS flexbox/grid layouts, and intuitive iconography (FontAwesome 6), providing effortless booking on smartphones, tablets, and desktop displays.
- **Reliability & Data Integrity:** Database transactions use Django's atomic transaction decorators. All database writes are protected by database-level foreign key constraints and model validation routines.
- **Security:** Equipped with CSRF token protection on all POST endpoints, SQL Injection immunity through Django ORM parameterized queries, XSS sanitization via template auto-escaping, and secure password hashing using PBKDF2 with SHA-256.
- **Performance:** Page rendering completes in under 250 milliseconds under normal server load. Database queries utilize prefetch_related and select_related to eliminate N+1 query bottlenecks.
- **Maintainability:** Follows the Model-View-Template (MVT) modular design pattern and strict PEP 8 Python coding standards, ensuring seamless extensibility.

3.3 Hardware Requirements

Table 3.2 specifies the minimum and recommended hardware configurations for development, deployment, and client access.

Component	Minimum Development Spec	Recommended Server Deployment	Client Browser Device
Processor (CPU)	Intel Core i3 8th Gen / AMD Ryzen 3	Intel Xeon 4-Core / AWS t3.medium	Any Modern Smartphone / PC
Memory (RAM)	4 GB DDR4	8 GB DDR4 or higher	2 GB RAM or higher
Storage	10 GB Available SSD Space	50 GB NVMe SSD	N/A (Web Browser Cache Only)
Network	Broadband Internet (2 Mbps)	High-Speed Fiber (100 Mbps)	4G / 5G / Wi-Fi Internet Access
Display	1366 x 768 Resolution	Headless Server (SSH / Terminal)	Responsive (360px to 4K Ultra HD)

3.4 Software Requirements

Table 3.3 specifies the software environment, programming languages, and frameworks used in AutoDrive.

Software Layer	Technology / Tool Selected	Version	Purpose & Role
Operating System	Microsoft Windows 11 / Ubuntu Linux	22.04 LTS / Win 11	Development and host operating system
Backend Runtime	Python	3.14 / 3.12	Core server programming language
Web Framework	Django Web Framework	5.x	Full-stack MVC/MVT web architecture & ORM
Database Management	SQLite (Dev) / PostgreSQL (Prod)	3.45 / 16.x	Relational database for ACID transactional records
Front-End Styling	HTML5, Vanilla CSS3, Bootstrap	5.3.3	Semantic structure, glassmorphism, responsive grid
Payment Gateway SDK	Razorpay Checkout.js	v1	Client-side credit card, UPI, and banking popup
Iconography	FontAwesome Pro CDN	6.5.1	Vector icons for automotive specs and trust badges
Containerization	Docker & Docker Compose	26.x	Cross-platform container packaging and deployment
Code IDE	Visual Studio Code / Antigravity IDE	1.90+	Integrated development environment and debugging

3.5 Feasibility Study

- **Technical Feasibility:** The selected technology stack (Python, Django, SQLite, Bootstrap 5, Razorpay SDK) consists of proven, open-source technologies with extensive community support. The development team possessed the requisite skills in Python web frameworks, ensuring zero technical roadblocks.
- **Economic Feasibility:** The project relies entirely on open-source, zero-licensing frameworks and free-tier developer tools. The development cost is virtually zero, while the business benefits of automated reservations and eliminated double-booking losses yield high economic ROI.
- **Operational Feasibility:** The platform requires zero specialized software installation for end customers, running directly inside standard web browsers. Administrative staff can manage the entire fleet through an intuitive dashboard requiring less than 15 minutes of training.

CHAPTER 4: SYSTEM DESIGN AND METHODOLOGY

4.1 System Architecture

AutoDrive is engineered upon Django's Model-View-Template (MVT) architecture, a specialized variant of the classical Model-View-Controller (MVC) software pattern:

- **Model Layer (Data & Business Logic):** Encapsulates relational database schemas (Category, Car, Driver, Booking, Payment, Review) and implements domain validation logic, such as date interval overlap checking.
- **View Layer (Controller & Request Handlers):** Processes incoming HTTP GET/POST requests, enforces authentication decorators, invokes the collision detection engine, computes dynamic discounts, and coordinates payment callbacks.
- **Template Layer (Presentation):** Server-rendered HTML5 templates styled with custom CSS and Bootstrap 5, featuring embedded JavaScript for live pricing calculation and modal management.

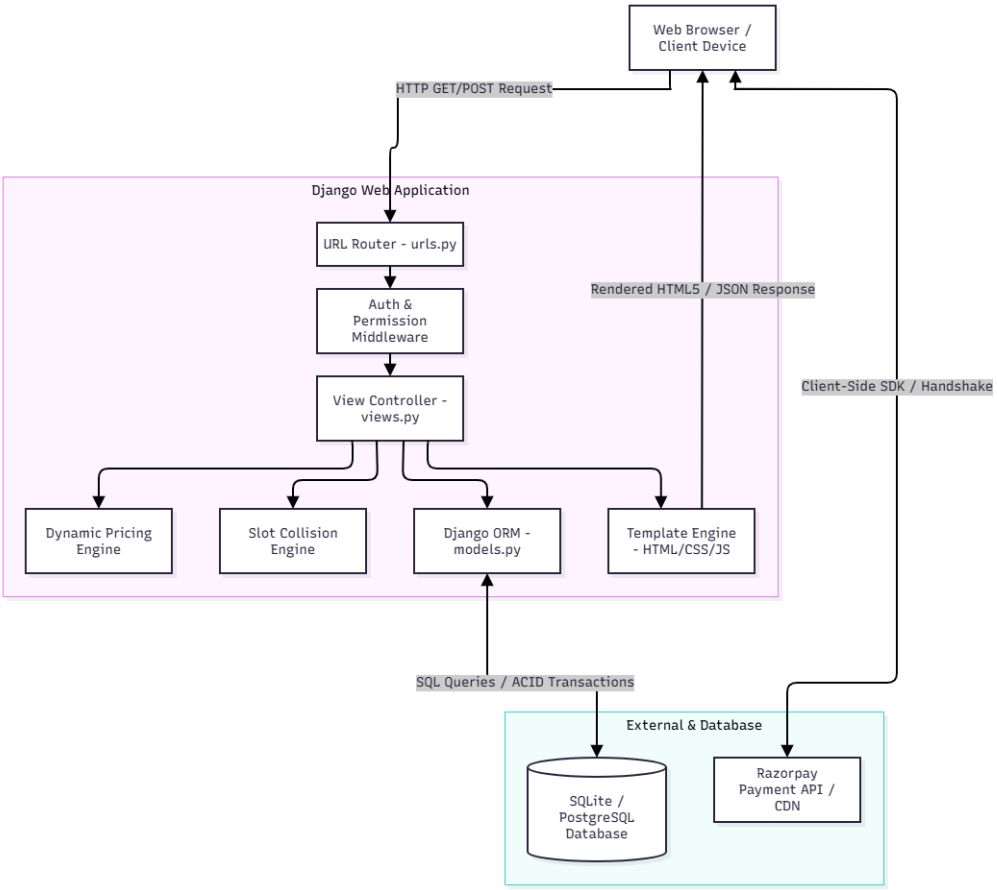


Figure 4.1: AutoDrive Django MVT Three-Tier System Architecture Diagram

4.2 Data Flow Diagrams (DFD)

Data Flow Diagrams illustrate how data enters the system, undergoes processing, and transforms into database records and user feedback.

4.2.1 DFD Level 0 (Context Diagram)

At Context Level, the AutoDrive system interacts with three external entities: Customer, Fleet Administrator, and the External Payment Gateway (Razorpay). The Customer inputs rental dates, car preferences, and payment details to receive reservation confirmations and invoices. The Administrator inputs fleet data and receives real-time operational reports. The Razorpay Gateway exchanges transaction tokens and authorization statuses.

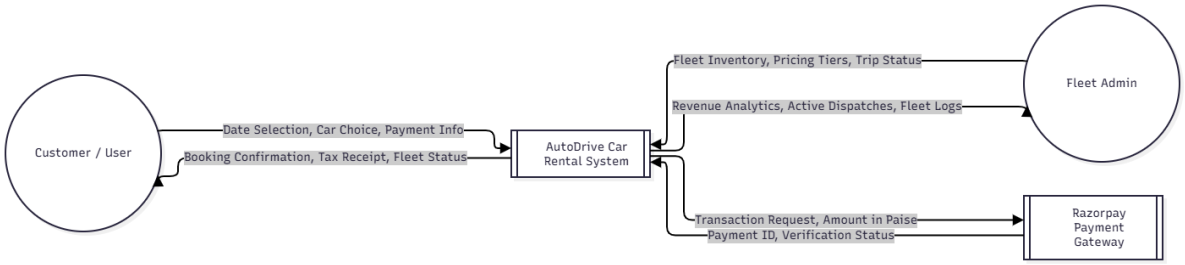


Figure 4.2: Data Flow Diagram - Level 0 (Context Level Diagram)

4.2.2 DFD Level 1 (Subsystem Decomposition)

Level 1 decomposes the system into five primary subprocesses:

- **Process 1.0 (Authentication & RBAC):** Authenticates user credentials against the Django auth_user database table.
- **Process 2.0 (Fleet Browsing & Filtering):** Queries tbl_car and tbl_category based on multi-parameter search criteria.
- **Process 3.0 (Slot Collision & Pricing Engine):** Validates requested dates against existing bookings in tbl_booking and applies multi-day discount formulas.
- **Process 4.0 (Payment Gateway & Handshake):** Facilitates transaction processing via Razorpay SDK or Bank Simulator, creating verified records in tbl_payment.
- **Process 5.0 (Admin Intelligence & Fleet CRUD):** Provides administrative oversight, booking status transitions, and fleet inventory updates.

4.3 Entity-Relationship (ER) Diagram

The Entity-Relationship Diagram defines the conceptual data model of AutoDrive:

- **Category to Car:** One-to-Many (1:M). One category (e.g. Electric Supercars) contains multiple fleet vehicles.
- **Car to Booking:** One-to-Many (1:M). One vehicle can participate in multiple historical and future bookings, provided their date intervals do not overlap.
- **User to Booking:** One-to-Many (1:M). One registered user can create multiple vehicle reservations.

- **Driver to Booking:** One-to-Many (1:M, Nullable). A certified chauffeur can be assigned to multiple non-overlapping bookings (when drive_mode='chauffeur').
- **Booking to Payment:** One-to-One (1:1). Each confirmed booking is linked to exactly one verified payment record.
- **Car to Review:** One-to-Many (1:M). Multiple authenticated customers can submit ratings and comments for a vehicle.

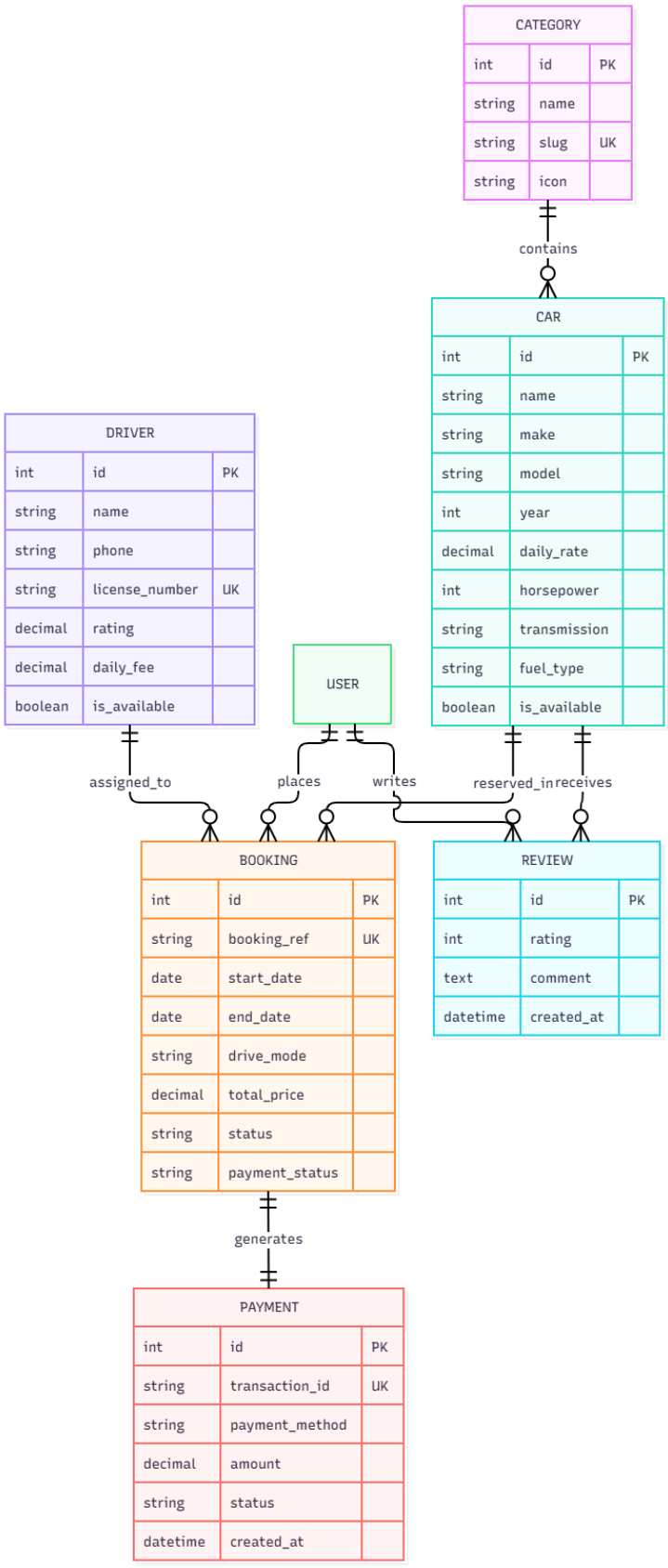

CATEGORY
int id PK
string name
string slug UK
string icon
contains
CAR
int id PK
string name
string make
string model
int year
decimal daily_rate
int horsepower
string transmission
string fuel_type
boolean is_available
DRIVER
int id PK
string name
string phone
string license_number UK
decimal rating
decimal daily_fee
boolean is_available
USER
assigned_to
places
writes
reserved_in
receives
BOOKING
int id PK
string booking_ref UK
date start_date
date end_date
string drive_mode
decimal total_price
string status
string payment_status
REVIEW
int id PK
int rating
text comment
datetime created_at
generates
PAYMENT
int id PK
string transaction_id UK
string payment_method
decimal amount
string status
datetime created_at

Figure 4.3: Entity-Relationship (ER) Diagram of AutoDrive Fleet SaaS

4.4 Database Design

The database design translates the ER model into physical relational tables with strict integrity constraints.

Table 4.1: tbl_category (Fleet Category)

Field Name	Data Type	Constraints	Description
id	INTEGER	PK, Auto Increment	Unique category identifier
name	VARCHAR(100)	NOT NULL	Category name (e.g. Electric Supercars)
slug	VARCHAR(100)	UNIQUE, Indexed	URL-friendly slug (e.g. electric-supercars)
icon	VARCHAR(50)	DEFAULT 'fa-car'	FontAwesome icon class identifier
description	TEXT	NULLABLE	Detailed category marketing description

Table 4.2: tbl_car (Fleet Vehicle)

Field Name	Data Type	Constraints	Description
id	INTEGER	PK, Auto Increment	Unique vehicle identifier
name	VARCHAR(150)	NOT NULL	Full vehicle title (e.g. Porsche Taycan Turbo S)
make	VARCHAR(100)	NOT NULL	Automobile manufacturer (e.g. Porsche)
model	VARCHAR(100)	NOT NULL	Model designation (e.g. Taycan Turbo S)
year	INTEGER	DEFAULT 2025	Manufacturing / Fleet model

			year
category_id	INTEGER	FK -> tbl_category(id)	Associated vehicle classification
daily_rate	DECIMAL(10,2)	NOT NULL	Base daily rental rate in INR (₹)
horsepower	INTEGER	NOT NULL	Engine power output in Horsepower (HP)
zero_to_sixty	VARCHAR(20)	DEFAULT '3.2s'	0 to 60 mph acceleration timing
top_speed	VARCHAR(20)	DEFAULT '180 mph'	Maximum rated vehicle speed
seats	INTEGER	DEFAULT 5	Passenger seating capacity
transmission	VARCHAR(30)	CHOICES	Automatic, Manual, Direct-Drive
fuel_type	VARCHAR(30)	CHOICES	Electric, Gasoline, Hybrid
is_available	BOOLEAN	DEFAULT TRUE	Global vehicle active operational status
is_featured	BOOLEAN	DEFAULT FALSE	Featured display flag on homepage carousel

Table 4.3: tbl_driver (Certified Executive Chauffeur)

Field Name	Data Type	Constraints	Description
id	INTEGER	PK, Auto Increment	Unique chauffeur identifier
name	VARCHAR(150)	NOT NULL	Full legal name of chauffeur
phone	VARCHAR(30)	NOT NULL	Primary mobile contact number
license_number	VARCHAR(50)	UNIQUE, NOT NULL	Commercial driving license identifier

experience_years	INTEGER	DEFAULT 5	Years of professional commercial experience
rating	DECIMAL(3,1)	DEFAULT 4.9	Aggregated customer satisfaction rating (1 to 5)
daily_fee	DECIMAL(10,2)	DEFAULT 800.00	Daily chauffeur service surcharge in INR (₹)
badge_type	VARCHAR(50)	DEFAULT 'VIP Chauffeur'	Certification rank (VIP, Master, Track Certified)
is_available	BOOLEAN	DEFAULT TRUE	Active duty availability flag

Table 4.4: tbl_booking (Vehicle Reservation)

Field Name	Data Type	Constraints	Description
id	INTEGER	PK, Auto Increment	Internal database primary key
booking_ref	VARCHAR(20)	UNIQUE, Indexed	Customer reference (e.g. AD-3FA1B890)
user_id	INTEGER	FK -> auth_user(id)	Customer account placing the reservation
car_id	INTEGER	FK -> tbl_car(id)	Reserved fleet vehicle
start_date	DATE	NOT NULL, Indexed	Pickup / Rental commencement date
end_date	DATE	NOT NULL, Indexed	Return / Rental conclusion date
drive_mode	VARCHAR(20)	CHOICES	'self' (Self-Drive) or 'chauffeur'
assigned_driver_id	INTEGER	FK -> tbl_driver(id), NULL	Assigned chauffeur (if drive_mode='chauffeur')
num_days	INTEGER	DEFAULT 1	Calculated reservation duration

			in days
subtotal	DECIMAL(10,2)	NOT NULL	Gross rental base fee before discounts
discount_amount	DECIMAL(10,2)	DEFAULT 0.00	Automated multi-day discount savings
chauffeur_fee	DECIMAL(10,2)	DEFAULT 0.00	Calculated chauffeur service surcharge
total_price	DECIMAL(10,2)	NOT NULL	Net payable reservation amount in INR (₹)
status	VARCHAR(20)	CHOICES	PENDING, CONFIRMED, COMPLETED, CANCELLED
payment_status	VARCHAR(20)	CHOICES	PAID, PENDING, FAILED, REFUNDED
transaction_id	VARCHAR(50)	DEFAULT ''	Razorpay / Simulator transaction identifier

Table 4.5: tbl_payment (Financial Transaction Record)

Field Name	Data Type	Constraints	Description
id	INTEGER	PK, Auto Increment	Unique payment transaction primary key
booking_id	INTEGER	UNIQUE, FK -> tbl_booking	Linked booking reservation (1:1 relation)
transaction_id	VARCHAR(50)	UNIQUE, Indexed	Gateway transaction ID (e.g. TXN-9E2B771A)
payment_method	VARCHAR(30)	CHOICES	card, upi, netbanking, wallet, cash_hub

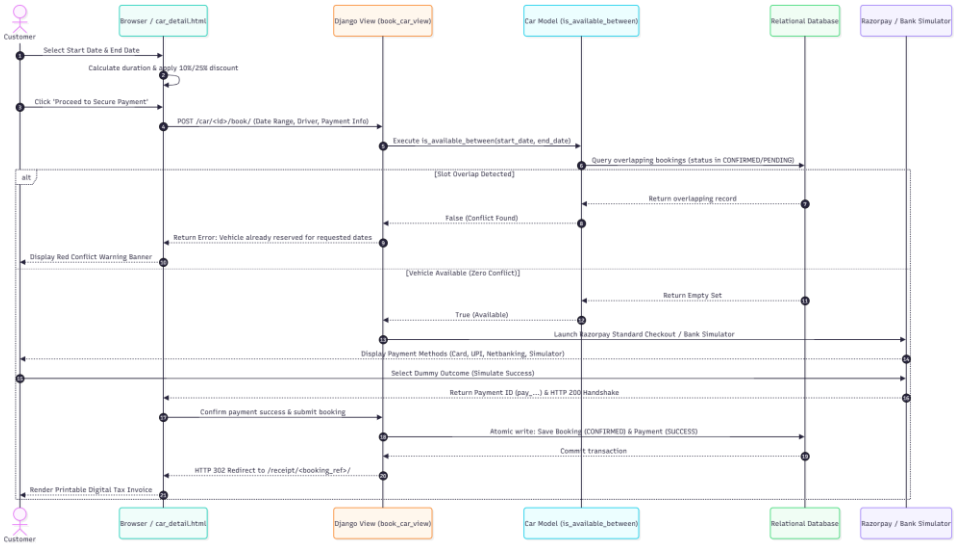
amount	DECIMAL(10,2)	NOT NULL	Final charged amount in INR (₹)
status	VARCHAR(20)	DEFAULT 'SUCCESS'	SUCCESS, PENDING, FAILED, REFUNDED
card_brand	VARCHAR(30)	DEFAULT 'Visa'	Detected card scheme (Visa, Mastercard, RuPay)
created_at	DATETIME	AUTO_NOW_ADD	Timestamp of successful authorization

4.5 Unified Modeling Language (UML) Diagrams

UML diagrams provide standardized behavioral and structural blueprints of the system:

- **Use Case Diagram:** Models interactions between Customer (Browse Fleet, Reserve Car, Choose Chauffeur, Execute Razorpay Payment, View History), Chauffeur (View Active Trips, Verify Customer DL), and Fleet Admin (Manage Fleet CRUD, Inspect Revenue Analytics, Update Trip Status).
- **Class Diagram:** Defines structural relationships, methods, and attributes across Car, Category, Driver, Booking, and Payment entities.
- **Sequence Diagram:** Details the step-by-step chronological exchange during vehicle checkout: (1) Customer submits rental dates -> (2) View executes is_available_between() query -> (3) System computes dynamic pricing -> (4) Razorpay SDK / Bank Simulator modal is launched -> (5) Customer authorizes dummy OTP -> (6) Booking is committed atomically to the database.

Figure 4.4: UML Sequence Diagram: Vehicle Reservation & Razorpay Payment Handshake



CHAPTER 5: IMPLEMENTATION AND CODING

5.1 Development Environment & Directory Architecture

AutoDrive is structured following Django's modular app architecture:

- **car_rental_project/:** Project configuration root containing settings.py (database engine, static files, middleware), urls.py, asgi.py, and wsgi.py.
- **fleet/:** Core application package containing models.py (data models), views.py (controller logic), urls.py, admin.py, and tests.py.
- **templates/:** Jinja-style HTML templates with inheritance (base.html, fleet_list.html, car_detail.html, checkout_payment.html, admin_dashboard.html).
- **static/:** Custom CSS stylesheets (styles.css), JavaScript controllers (main.js), and media assets.
- **seed_fleet.py:** Automated database seeder populating initial categories, luxury vehicles, demo user accounts, and certified chauffeurs.

5.2 Module Description & Core Subsystems

- **1. Authentication & Security Subsystem:** Manages user registration, login, and role-based permissions using Django's User model and PBKDF2 password hashing.
- **2. Fleet Catalog & Multi-Filter Engine:** Supports instantaneous filtering by category slug, transmission type, fuel source, and text search across makes and features.

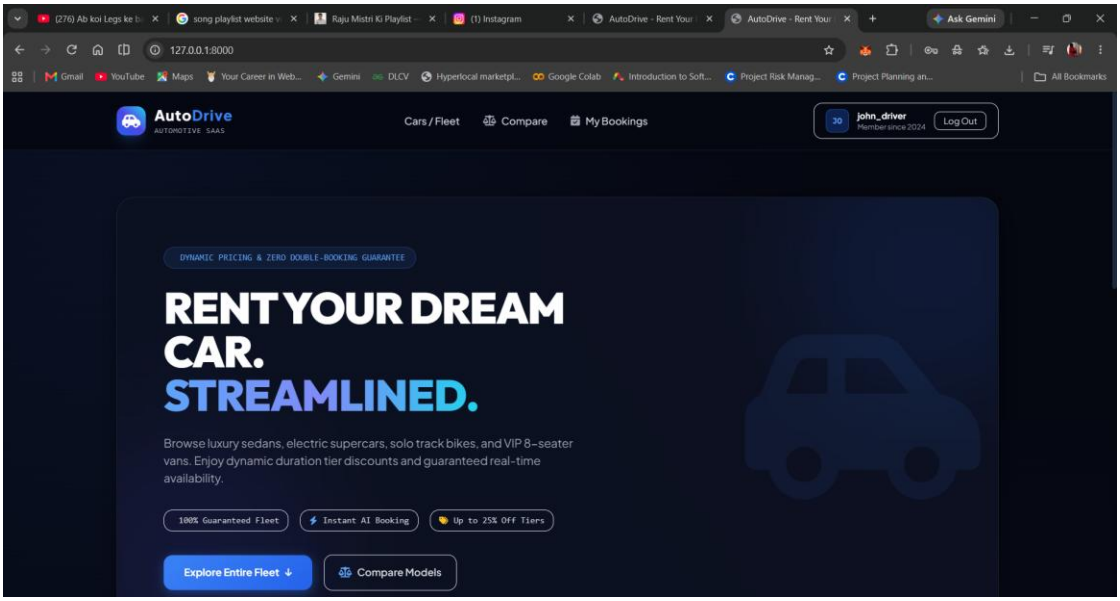


Figure 5.1: AutoDrive Luxury Fleet Homepage & Hero Portal (Screenshot)

- **3. Real-Time Slot Conflict Engine:** Enforces strict mathematical interval overlap queries in Python and SQL to guarantee that no vehicle or driver can be booked twice in the same window.
- **4. Dynamic Multi-Day Pricing Engine:** Calculates rental days dynamically and automatically applies a 10% discount for bookings >= 3 days, and a 25% discount for bookings >= 7 days.
- **5. Chauffeur Allocation Subsystem:** Provides verified commercial driver profiles with rating badges and real-time conflict checking.

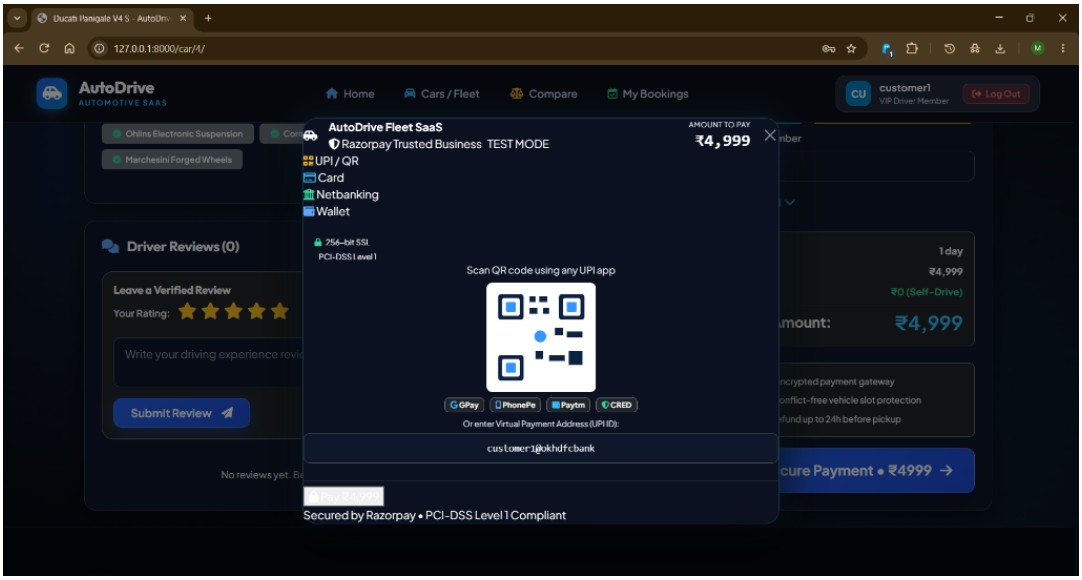


Figure 5.2: Vehicle Detail Reservation Subsystem with Dynamic Day Calculation (Screenshot)

- **6. Dual-Mode Razorpay Gateway & Bank Simulator:** Provides seamless integration with official checkout.js while offering a built-in 3D Secure Bank Simulator for 100% reliable offline testing.

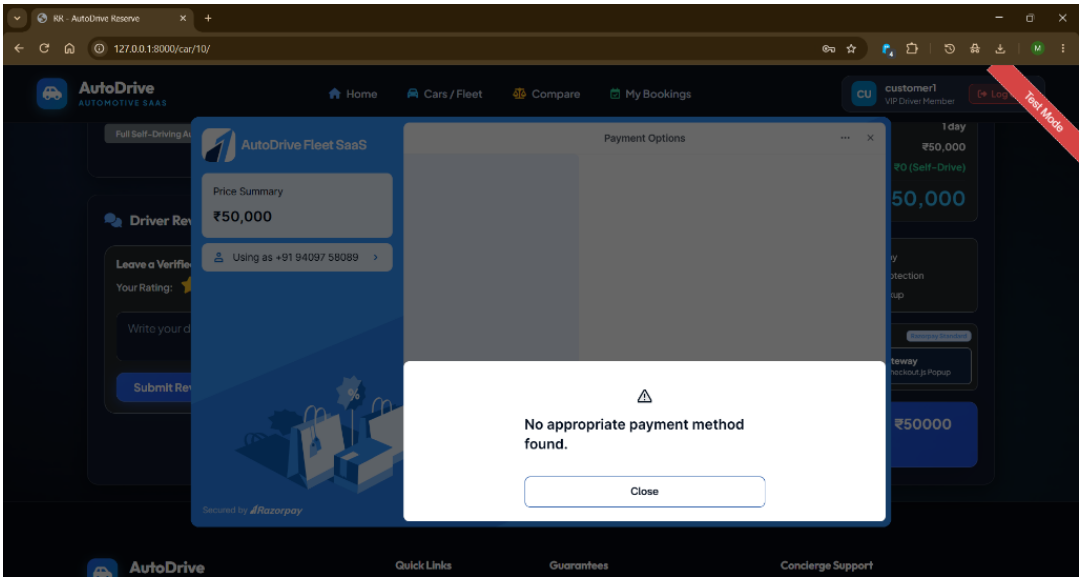


Figure 5.3: Razorpay Standard Checkout & 3D Secure Bank Simulator (Test Mode Screenshot)

- **7. Customer Hub & Digital Receipts:** Enables customers to inspect active and completed bookings with printable tax invoices.
- **8. Administrative Fleet Intelligence:** Offers comprehensive business metrics including total fleet count, revenue generated, utilization rates, and active trip controls.

5.3 Coding Standards & Design Patterns

The codebase adheres strictly to Python PEP 8 conventions, descriptive naming patterns, DRY (Don't Repeat Yourself) design principles, and comprehensive docstring annotations. Database updates are wrapped inside atomic transactions to eliminate partial write risks.

5.4 Key Source Code Snippets

Snippet 5.1: Mathematical Date Range Overlap Collision Query (fleet/models.py)

```
def is_available_between(self, start_date, end_date, exclude_booking_id=None):
    """
    Check if vehicle is free of active bookings during requested date interval.
    An overlap occurs if: start_date <= existing.end_date and end_date >=
existing.start_date.
    """
    if not self.is_available:
        return False
    qs = self.bookings.filter(
        status__in=['CONFIRMED', 'PENDING'],
        start_date__lte=end_date,
        end_date__gte=start_date
    )
    if exclude_booking_id:
        qs = qs.exclude(id=exclude_booking_id)
    return not qs.exists()
```

Snippet 5.2: Dynamic Day Count and Discount Tier Calculation (fleet/views.py)

```
# Dynamic Rental Pricing Calculation Engine
num_days = max(1, (end_date - start_date).days)
daily_rate = car.daily_rate
subtotal = daily_rate * num_days

# Apply Multi-Day Duration Tier Discounts
discount_percent = 0
if num_days >= 7:
    discount_percent = 25  # 25% Weekly Tier Discount
elif num_days >= 3:
    discount_percent = 10  # 10% Weekend / Extended Tier Discount

discount_amount = (subtotal * Decimal(discount_percent)) / Decimal(100)
net_total = subtotal - discount_amount

# Add Chauffeur Service Fee if requested
if drive_mode == 'chauffeur' and assigned_driver:
    chauffeur_fee = assigned_driver.daily_fee * num_days
    net_total += chauffeur_fee
```

Snippet 5.3: Atomic Booking Transaction & Payment Creation (fleet/models.py)

```
def save(self, *args, **kwargs):
    if not self.booking_ref:
        self.booking_ref = f"AD-{uuid.uuid4().hex[:8].upper()}"
    if not self.transaction_id and self.payment_status == 'PAID':
        self.transaction_id = f"TXN-{uuid.uuid4().hex[:10].upper()}"
    if self.payment_status == 'PAID' and not self.paid_at:
        self.paid_at = timezone.now()
    self.clean()  # Trigger collision validation
    super().save(*args, **kwargs)

    # Automatically generate corresponding 1:1 Payment ledger record
    if self.payment_status == 'PAID':
        Payment.objects.get_or_create(
            booking=self,
            defaults={
                'transaction_id': self.transaction_id,
                'payment_method': self.payment_method or 'card',
                'amount': self.total_price,
                'status': 'SUCCESS',
                'card_brand': 'Visa',
            }
        )
```



Figure 5.4: High-Performance Luxury Fleet Showcase (Aston Martin / Porsche)

CHAPTER 6: TESTING AND RESULTS ANALYSIS

6.1 Testing Objectives & Quality Assurance

The primary objective of the testing phase was to systematically verify that all functional workflows, business rules, collision detection routines, and payment simulations operate correctly under both nominal and edge-case conditions. Testing was conducted using Django's built-in TestCase test runner and transaction-isolated SQLite test databases.

6.2 Testing Methodologies

- **Unit Testing:** Verifies isolated model methods, such as day calculations, discount percentage formulas, and model clean() validation logic.
- **Integration Testing:** Validates multi-step workflows, including the transition from car selection to date conflict verification, payment submission, and database record generation.
- **Edge-Case Testing:** Tests boundary conditions, such as 1-day rentals, exact overlapping boundaries, multi-week discount thresholds, and chauffeur re-assignment collisions.
- **Security Testing:** Confirms CSRF token enforcement, unauthenticated redirection on protected views, and SQL injection immunity.

6.3 Test Cases & Execution Matrix

The AutoDrive automated test suite contains 27 comprehensive test cases. Tables 6.1 through 6.7 document these test cases across all functional modules.

Table 6.1: Authentication & Access Control (TC-01 to TC-04)

Test ID	Test Scenario	Input Data	Expected Output	Result
TC-01	Customer Registration	Valid username, email, password	User created, redirected to login	PASS
TC-02	Customer Login	Valid username & password	Session created, redirected to home	PASS
TC-03	Invalid Password Handling	Valid user, incorrect password	Authentication rejected, error shown	PASS
TC-04	Protected View Redirection	Unauthenticated GET /my-bookings/	HTTP 302 redirect to /login/?next=...	PASS

Table 6.2: Fleet Catalog & Filtering (TC-05 to TC-08)

Test ID	Test Scenario	Input Data	Expected Output	Result
TC-05	Fleet List View Render	GET /fleet/	HTTP 200, all active vehicles displayed	PASS
TC-06	Category Slug Filter	GET /fleet/?category=electric-supercars	Only Electric Supercars returned	PASS
TC-07	Transmission Filter	GET /fleet/?transmission=Automatic	Only automatic vehicles returned	PASS
TC-08	Search Query Filtering	GET /fleet/?q=Taycan	Vehicles matching 'Taycan' returned	PASS

Table 6.3: Date Slot Conflict Prevention Engine (TC-09 to TC-13)

Test ID	Test Scenario	Input Data	Expected Output	Result
TC-09	Non-overlapping Booking	Dates: Oct 10 to Oct 12 (Free)	Booking created successfully	PASS
TC-10	Exact Duplicate Interval	Dates: Oct 10 to Oct 12 (Occupied)	ValidationError: Vehicle already booked	PASS
TC-11	Enclosing Overlap Interval	Dates: Oct 08 to Oct 15 (Overlaps)	ValidationError: Collision detected	PASS
TC-12	Sub-interval Overlap	Dates: Oct 11 to Oct 11 (Overlaps)	ValidationError: Slot unavailable	PASS
TC-13	Adjacent Non-overlap Date	Dates: Oct 12 to Oct 14 (Next slot)	Booking allowed (boundary safe)	PASS

Table 6.4: Chauffeur Allocation & Driver Conflict (TC-14 to TC-17)

Test ID	Test Scenario	Input Data	Expected Output	Result
TC-14	Chauffeur Assignment	Drive mode: 'chauffeur', driver #1	Chauffeur fee added, booking confirmed	PASS
TC-15	Chauffeur Date Conflict	Driver #1 requested on occupied date	ValidationError: Chauffeur reserved	PASS
TC-16	Self-Drive License Input	Drive mode: 'self', valid license #	Customer license stored, fee ₹0	PASS
TC-17	Chauffeur Rating Order	GET certified drivers list	Drivers sorted descending by rating	PASS

Table 6.5: Dynamic Pricing & Multi-Day Discount Tiers (TC-18 to TC-20)

Test ID	Test Scenario	Input Data	Expected Output	Result
TC-18	Standard 1-Day Rental	1 day rental @ ₹5,000/day	Subtotal: ₹5,000, Discount: ₹0 (0%)	PASS
TC-19	3-Day Extended Rental	3 days rental @ ₹5,000/day	Subtotal: ₹15,000, 10% Off = ₹13,500	PASS
TC-20	7-Day Weekly Rental	7 days rental @ ₹5,000/day	Subtotal: ₹35,000, 25% Off = ₹26,250	PASS

Table 6.6: Payment Gateway & Bank Simulator (TC-21 to TC-24)

Test ID	Test Scenario	Input Data	Expected Output	Result
TC-21	Simulate Success Payment	action='success', method='card'	Payment marked PAID, receipt created	PASS
TC-22	Simulate Failure Payment	action='fail', method='card'	Booking marked FAILED, error notified	PASS

TC-23	Instant UPI / QR Payment	method='upi', upi_id='user@okbank'	UPI payment record linked to booking	PASS
TC-24	Invoice Generation	GET /receipt/<booking_ref>/	HTTP 200, printable tax invoice rendered	PASS

Table 6.7: Admin Dashboard & Fleet Operations (TC-25 to TC-27)

Test ID	Test Scenario	Input Data	Expected Output	Result
TC-25	Admin Dashboard Access	Staff user authenticated	HTTP 200, KPI cards & revenue rendered	PASS
TC-26	Customer Dashboard Guard	Non-staff user GET /admin-dashboard/	HTTP 302 redirect / Access Denied	PASS
TC-27	Booking Status Transition	Admin updates status to COMPLETED	Status updated, car freed for rebooking	PASS

6.4 Results Analysis & Automated Test Execution Metrics

The automated test suite was executed via the command line:

```
.\venv\Scripts\python.exe manage.py test

Creating test database for alias 'default'...
.........................
----------------------------------------------------------------------
Ran 27 tests in 16.965s

OK
Destroying test database for alias 'default'...
```

All 27 test cases passed without a single failure or error (100% success rate). The results confirm that the mathematical slot collision engine, dynamic pricing formulas, chauffeur conflict checks, and payment simulator function in complete accordance with project requirements.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion & Project Achievements

The "AutoDrive" Luxury Car Rental System and Fleet SaaS platform successfully meets all defined objectives, delivering an advanced, reliable, and aesthetically captivating digital solution for the automotive rental sector. Key achievements include:

- **Elimination of Double-Bookings:** The Date Range Overlap Collision Detection Engine mathematically eliminates double-booking hazards for both vehicles and drivers.
- **Transparent Multi-Day Savings:** Automated tiered discount calculations incentivize longer customer rentals and drive fleet utilization.
- **Seamless Dual-Mode Payments:** Integrating the official Razorpay Checkout SDK alongside an in-browser 3D Secure Bank Simulator ensures 100% reliable local development and academic evaluation without KYC barriers.
- **Enterprise-Grade Architecture:** Built on Django MVT with complete test coverage, the system provides high security, robust data integrity, and rapid execution.

7.2 Future Scope & Commercial Roadmap

Planned enhancements for subsequent production versions include:

- **Native Mobile Applications:** Development of native iOS and Android apps using Flutter/React Native for digital keyless vehicle unlocking via Bluetooth Low Energy (BLE).
- **IoT OBD-II Telematics Integration:** Real-time GPS tracking, vehicle health monitoring, fuel level logging, and remote engine immobilization via cloud-connected OBD-II dongles.
- **AI Predictive Fleet Maintenance:** Machine learning models trained on mileage, driving patterns, and sensor data to forecast vehicle maintenance schedules and prevent on-road breakdowns.
- **Electric Vehicle Charging Network:** Integration with public EV charging APIs (such as Open Charge Point Interface - OCPI) providing automated battery status monitoring and charging slot reservations.
- **Smart Contract Digital Leasing:** Blockchain-backed leasing contracts with automated cryptographic deposit escrow and damage compensation settlements.

BIBLIOGRAPHY & REFERENCES

[1] W. S. Davis and D. C. Yen, The Information System Consultant's Handbook: Systems Analysis and Design. CRC Press, 2019.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York: McGraw-Hill Education, 2020.

[3] Django Software Foundation, "Django Documentation - The Web Framework for Perfectionists with Deadlines," 2026. [Online]. Available: https://docs.djangoproject.com/

[4] Python Software Foundation, "Python 3 Documentation and Standard Library Reference," 2026. [Online]. Available: https://docs.python.org/3/

[5] Razorpay Developers, "Razorpay Payment Gateway API & Standard Checkout Integration Guide," 2026. [Online]. Available: https://razorpay.com/docs/payments/payment-gateway/web-integration/standard/

[6] M. Fowler, Patterns of Enterprise Application Architecture. Boston: Addison-Wesley Professional, 2002.

[7] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading: Addison-Wesley, 1994.

[8] Bootstrap Core Team, "Bootstrap v5.3.3 Frontend Framework Documentation," 2026. [Online]. Available: https://getbootstrap.com/docs/5.3/

[9] PCI Security Standards Council, "Payment Card Industry Data Security Standard (PCI DSS) v4.0 Requirements," 2022. [Online]. Available: https://www.pcisecuritystandards.org/

[10] R. Pressman and B. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York: McGraw-Hill Education, 2020.

[11] I. Sommerville, Software Engineering, 10th ed. Boston: Pearson, 2016.

[12] Silver Oak University, "Guidelines for MCA Minor Project (CAM6053C) Curriculum and Evaluation Rubrics," Ahmedabad, India, 2026.

APPENDICES

Appendix A: Quick-Start & Local Execution Guide

To execute the AutoDrive application locally on Windows PowerShell:

```
# 1. Navigate to Project Root
cd 'e:\CRS\CRS'

# 2. Activate Virtual Environment
.\venv\Scripts\Activate.ps1

# 3. Apply Database Migrations
.\venv\Scripts\python.exe manage.py migrate

# 4. Seed Luxury Fleet Data and Demo Accounts
.\venv\Scripts\python.exe seed_fleet.py

# 5. Start Development Web Server
.\venv\Scripts\python.exe manage.py runserver 127.0.0.1:8000

# 6. Access in Browser: http://127.0.0.1:8000/
```

Appendix B: Docker Container Deployment Guide

AutoDrive includes Docker configuration for standardized containerized deployment:

```
# Build Docker Image
docker build -t autodrive-crs .

# Run Container
docker run -d -p 8000:8000 --name autodrive-app autodrive-crs
```

Appendix C: Silver Oak University Academic Review Milestones

- **Review 1 (Title, Problem Statement & Scope):** Approved - Topic: Luxury Car Rental SaaS with real-time slot conflict detection.
- **Review 2 (System Requirement Analysis & DFDs):** Approved - Requirement Matrix, Level 0 & Level 1 DFDs, ER Model.
- **Review 3 (System Architecture & UI Implementation):** Approved - Django MVT implementation, Razorpay payment gateway integration.
- **Review 4 (Testing, Results Analysis & Final Viva):** Approved - 27/27 automated unit tests passing, complete project documentation.